

Week-7 Assignment: Delta Lake MERGE Implementation

Incremental Data Processing using Delta Lake (MERGE Operation)

Dataset: Sample Superstore Dataset

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopargaon – 423603

E-mail: snehalbhosale1807@gmail.com

CEI ID: CT_CSI_DE_1177

Technology Used

- Apache Spark (PySpark)
- Delta Lake
- Python
- Google Colab / Jupyter Notebook
- Spark SQL
- CSV File Format
- Delta Table

1. Objective

The objective of this assignment is to understand the fundamentals of Delta Lake by implementing an incremental data processing pipeline using the MERGE operation. The assignment focuses on loading data into a Delta table, performing data cleaning, simulating incremental data changes, applying UPSERT (Update + Insert) logic using the MERGE command, validating the final dataset, and demonstrating the advantages of Delta Lake for reliable and efficient data management.

2. Introduction

Traditional data lakes built on plain file formats such as CSV or Parquet do not support reliable update or delete operations. Any change to existing data usually requires rewriting entire files, which is slow, error-prone, and unsuitable for real-world data pipelines where data keeps changing every day. Delta Lake solves this problem by adding a transaction log on top of Parquet files, which brings ACID properties, schema enforcement, and support for update, delete, and merge operations directly on the data lake.

In real organizations, data rarely arrives all at once. A source system usually sends a full data load initially, called the master or base dataset, and then keeps sending small batches of new or changed records afterward, called incremental data. This assignment simulates exactly that situation using the Sample Superstore dataset. A master dataset was created first, an incremental dataset with a few modified and a few brand-new records was generated afterward, and Delta Lake's MERGE operation was used to combine both correctly, without creating duplicate records or losing any existing data.

3. Implementation Steps

The notebook was implemented step by step in Google Colab, following an industry-style incremental ETL pattern. The key steps are summarized below.

1. Install Required Libraries – Installed Java, PySpark, and Delta Lake dependencies needed to run Spark with Delta support in Colab.
2. Import Required Libraries – Imported SparkSession, Spark SQL functions, and Delta Lake modules.
3. Initialize SparkSession with Delta Lake – Created a Spark session with Delta Lake extensions enabled.
4. Upload and Load Dataset – Uploaded the Sample - Superstore.csv file and loaded it into a Spark DataFrame.
5. Explore Dataset – Checked the total number of records, columns, and the schema of the dataset.
6. Data Cleaning – Removed duplicate rows and dropped records containing null values.
7. Validate Cleaned Data – Verified duplicate Order IDs and confirmed there were no remaining null values.
8. Create Master Dataset – Took a fixed 80 percent of the cleaned data, sorted for reproducibility, as the master dataset.
9. Create Incremental Dataset – Took the remaining 20 percent of the cleaned data as the incremental dataset.
10. Create Delta Table – Wrote the master dataset into a Delta table stored at delta/superstore.
11. Simulate Incremental Changes – Modified the Sales value of existing records and added three brand-new order records to imitate real incoming data.
12. Perform MERGE Operation – Used DeltaTable.merge() with Row_ID as the join key to update matched records and insert unmatched ones in a single atomic operation.

13. Read Updated Delta Table – Reloaded the Delta table after the merge to confirm the changes were applied.
14. Validate Results – Rechecked the final row count, duplicate values, and null values after the merge.
15. Display Final Dataset Summary – Displayed total records, unique orders, and unique customers as a final summary.

4. Working Flow of the Pipeline

The overall pipeline followed a simple and logical flow, similar to how incremental loading works in real data engineering systems. The flow can be summarized as follows.

4.1 Data Ingestion and Cleaning

The raw Superstore CSV file was loaded into a Spark DataFrame. Duplicate rows were dropped using `dropDuplicates()`, and rows with missing values were removed using `dropna()`. This produced a clean DataFrame that was used for all later steps.

4.2 Master and Incremental Split

Instead of using a random split, the cleaned dataset was sorted and then divided into a fixed 80 percent master portion and a 20 percent incremental portion using `limit()` and `subtract()`. This kept the notebook fully reproducible, so the same records appear in the master and incremental datasets every time the notebook is run.

4.3 Delta Table Creation

The master dataset was written to disk in Delta format at the path `delta/superstore`. This created the transaction log folder (`_delta_log`) along with the underlying Parquet data files, which together make up a Delta table.

4.4 Simulating Incremental Data

To imitate a real incoming data batch, the Sales column of the incremental records was increased by a fixed amount to represent updated values, and three completely new order records were created and appended. This gave a combined incremental batch containing both updates and new inserts.

4.5 MERGE Operation (Upsert)

The Delta table was loaded using `DeltaTable.forPath()`, and the `merge()` method was used with `Row_ID` as the matching key. The `whenMatchedUpdateAll()` clause updated existing rows whose `Row_ID` matched an incoming record, and `whenNotMatchedInsertAll()` inserted rows that did not already exist. This single MERGE statement performed both the update and insert logic in one atomic step.

4.6 Validation

After the merge, the Delta table was reloaded and checked for the total row count, duplicate Order IDs, and null values, to confirm the data remained clean and consistent after the upsert.

5. Results

The table below summarizes the key results obtained after running the pipeline end to end.

Metric	Value
Records before cleaning	9994
Records after cleaning	9994
Master dataset size (80%)	7995
Incremental dataset size (20%)	1999
Final row count after MERGE	9997
Unique orders after MERGE	5012
Unique customers after MERGE	796
Null values after MERGE	0

The final row count of 9997 correctly reflects the original 9994 cleaned records plus the 3 new order records inserted through the MERGE operation, while the existing matched records were updated in place. No null values were found after the merge, confirming that the pipeline preserved data integrity throughout.

6. Advantages of Delta Lake Observed

- ACID transactions ensured the MERGE operation either completed fully or not at all, avoiding partial or corrupted writes.
- A single MERGE statement handled both update and insert logic, removing the need for separate update and insert jobs.
- Schema enforcement prevented mismatched or malformed data from silently corrupting the Delta table.
- The underlying transaction log made it possible to track every write operation made to the table.
- Delta Lake worked directly on top of existing Parquet files, requiring no separate database system.

7. Conclusion

In this assignment, the Sample Superstore dataset was loaded into a Delta table and cleaned by removing duplicate and null records. A fixed master and incremental dataset were created to keep the workflow reproducible. Using Delta Lake's MERGE operation, existing records were updated and new records were inserted in a single, reliable step. Validation after the merge confirmed the integrity of the final dataset through row count, duplicate checks, and null value verification. Overall, this implementation demonstrates a complete and reliable incremental ETL pipeline using Delta Lake, and reflects how real-world data engineering teams manage continuously changing data in a data lake environment.